

MapReduce 架構下的平行化局部診斷設計與模擬

林泰毅

國立東華大學資訊工程系

talesfan0502@gmail.com

賴寶蓮

國立東華大學資訊工程系

baolein@mail.ndhu.edu.tw

許弘駿

慈濟大學醫學資訊系

hchsu@mail.tcu.edu.tw

摘要

分散式系統中，如何即時、正確、有效率的診斷(diagnosis)出壞掉的處理器，是一項重要議題。局部診斷演算法，可診斷任意一點或一小部分的點(處理器)，但若對一個大型網路結構做全面性診斷，須診斷的點數和輸入資料都會非常龐大。

MapReduce 是利於處理大規模資料的分散式程式框架，適合與局部診斷演算法結合，而在所有可運行 MapReduce 的平台中以開放原始碼的 Hadoop 最被廣為研究，因此本論文基於局部診斷演算法，設計 2 種在 Hadoop 平台上的症狀平行模擬程式，分別以常見的規則圖，如超立方體(hypercube)、交錯立方體(crossed Cube)以及雙扭立方體(twisted Cube)結構網路等，與隨機簡單圖為診斷對象，我們提供在 Hadoop 平台上模擬診斷的實驗數據以呈現局部診斷演算法的平行運算效率。

關鍵字: 分散式系統、局部診斷(local diagnosis)、MapReduce、超立方體(hypercube)、交錯立方體(crossed cube)、雙扭立方體(twisted cube)、隨機簡單圖

一. 簡介

由於高階伺服器及超級電腦的造價昂貴，許多大型運算紛紛採用分散式系統進行運算，因為藉由網路連結大量平價電腦的分散式系統或許能夠以更少的成本達到相同的效益；而由多電腦所構築的系統裡，無法保證所有電腦都能正常運行，隨時可能有電腦故障，因此如何即時、正確、有效率的診斷出壞掉的電腦，是一項重要議題。

影響分散式系統效能的因素，除了硬體之外，網路結構亦相對重要；在分散式系統中一台電腦可看作一個節點，節點之間藉由網路進行傳輸與溝通，而以具有良好特性的圖形結構作為分散式系統的基底，可使資料傳輸(data transmission)、廣播(broadcasting)、路由選擇

(routing)的成本降低，在進行壞點(faulty node)診斷時，可以簡單的規則及較好的效率找出壞點，而超立方體和以其為基礎所改良的一些圖形，為較常被選用的分散式系統網路架構。

一個圖形若為 t -可診斷(t -diagnosable)[9]，則在壞點數量小於或等於 t ($\leq t$) 的情況下都能正確診斷出所有壞點，因此若要針對某個圖形結構設計診斷演算法，需先得知該圖形的整體可診斷度(diagnosability) t 是多少之後，才能設計演算法並判斷在各種情境下可否正確診斷，目前大部分的演算法會一次測試全圖以確認所有點的好壞，但無法單就圖中某個區塊或某個點進行診斷；另外，假設某一圖形 G 可診斷度是 t ，則當壞點數量是 $t+1$ 時無法正確找出所有壞點，但不代表 G 中所有點都無法確認好壞，假設將 G 切割成 A 、 B 兩個區塊後， A 區塊的可診斷度是 $t+1$ 、 B 區塊是 t ， A 區塊其實能容忍更高的壞點數，然而以圖 G 的整體可診斷度來看則會忽視 A 區塊較佳的可診斷度，因此當 A 和 B 兩區塊的可診斷度差距很大時，為求圖形的整體可診斷度而忽略 A 區各點的可診斷度，將造成 A 區可診斷力的浪費。

t -局部可診斷(local t -diagnosable)的概念[4]指的是圖中單一節點的可診斷度，以此概念所設計的局部診斷演算法縮小了診斷範圍，能夠針對單一點或者特定區域的點做診斷，可將每個點的可診斷力各自發揮到極限，診斷力高的點不再受診斷力差的點拘束。

執行診斷演算法的平台可以是單一電腦或者分散式系統，亦能運用雲端運算平台來處理；以效率來看，單一電腦需獨力診斷系統中全部節點，運算與資料皆無法分攤，而分散式系統可將診斷工作平均分配給各節點，再把所有節點的診斷結果回收到一台電腦之中，其缺點是會有節點之間交換資料的時間浪費，且傳輸資料受網路速度限制，單一電腦沒有這個問題，儘管如此，當診斷規模龐大時，單一電腦跟分散式的處理速度將清晰可辨。Hadoop 是雲端運算平台之一，基於分散式系統之上且是單主多僕(master-slaves)的架構，它擁有專屬的分散式檔案系統(HDFS)，可快速存取大量的資料，以及專屬的分散式資料庫(HBase)，它記錄一組組的鍵值組對(key-value pair)，去除了傳統關聯式

資料庫繁雜的資料型態與關聯性，還有提供 MapReduce API，它擁有許多切割資料與分割運算的方法，開發者不需要再耗費太多時間於工作分割的程式設計，目前已有許多問題被以 MapReduce 方式來實作 [7][11][13]；Hadoop 因開放原始碼的緣故，至今已被廣泛研究與討論，所以已擁有很多相關研究資源可供使用，因此相較於其他雲端平台，Hadoop 在研究與開發上會較容易。

本論文致力於局部診斷演算法於 Hadoop 平台上的模擬，首先以超立方體、交錯立方體、以及雙扭立方體為診斷對象，接著以隨機簡單圖為對象，記錄診斷時間與結果，並為兩種模擬程式作分析。

二. 局部診斷

超立方體是已被廣泛研究的一種圖形，它擁有良好的結構與特性，例如對稱性(symmetry)、較低的直徑(diameter)、遞迴建構性，並且已有廣播和路由選擇的演算法可以使用，因此常被作為分散式系統的網路架構，而以超立方體為基礎所改良的圖形也很多，其中較有名的是交錯立方體[2]和雙扭立方體[3]，它們一樣擁有良好的特性，並常被作為基底的網路架構，這些立方體已有許多診斷的相關研究[1][8][12]；我們把 n 維的超立方體記為 Q_n ，它擁有 2^n 個頂點、 $n2^{n-1}$ 條邊、連通度(connectivity)和直徑皆為 n ，每個節點都擁有 n 個位元所組成的獨一無二的編號，且分支度皆為 n ；連線規則也相當單純，相鄰的兩點恰好一個位元相異，圖 1 為 Q_3 的圖形結構。

當進行壞點診斷時，須先確立測試者(tester)和受測者(testee)，它們各代表一個節點，且兩點必須要直接連線；進行測試時，測試者傳送信息給受測者，受測者接收信息後，回傳信息給測試者，測試者再根據傳回的信息，判斷受測者的狀態，決定測試結果；實際測試方式與使用的診斷模型有關，目前已有許多網路診斷模型，較常被應用的是 PMC 模型(PMC Model) [9]。PMC 模型的規則是，當測試者和受測者都是好點，則測試後結果記為 0，若測試者是好點受測者是壞點，則結果記為 1，如果測試者是壞點，那麼無論受測者的好壞，結果均可能為 0 或 1，測試規則如表 1；當圖形上所有節點互相測試完成後，收集所有到的測試結果，我們稱為症狀(symndrome)，症狀將是系統診斷的依據。

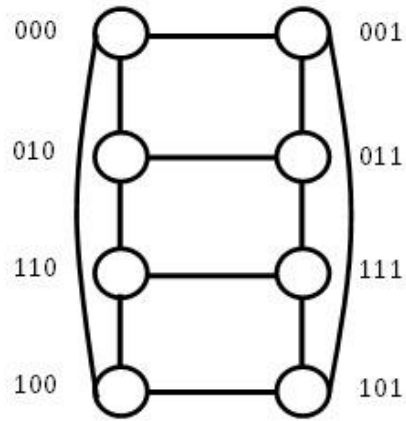


圖 1. Q_3 的圖形結構

測試者	受測者	測試結果
好	好	0
好	壞	1
壞	好	0 或 1
壞	壞	0 或 1

表 1. PMC 模型的測試規則

局部診斷演算法在診斷圖中任意一點時，會先計算該點的局部可診斷度，Hsu 等人[4]提出一個結構稱為 Type I，簡稱 T_1 ，在使用 PMC 模型的情況下它讓局部診斷演算法能確立圖形中任意一點的局部可診斷度， T_1 的結構定義以及定理如下所示。

定義 1[4]. $G(V, E)$ 是一個圖形，設 v 是 G 中任意一點， k 是一個正整數，且 $k \geq 1$ ，以 v 為根點， k 條分支的 Type I 結構 $T_1(v; k)$ 定義如下：

$$T_1(v; k) = |V(v; k), E(v; k)|$$

$T_1(v; k)$ 擁有 $2k + 1$ 個點和 $2k$ 條邊，如圖 2，其點與邊的定義如下：

- $V(v; k) = \{v\} \cup \{x_i, y_i | 1 \leq i \leq k\}$
- $E(v; k) = \{(v, x_i), (x_i, y_i) | 1 \leq i \leq k\}$

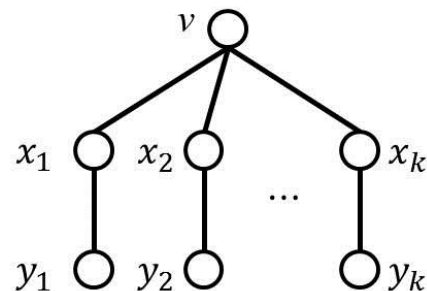


圖 2. $T_1(v; k)$ 的結構

定理 1[4]. $G(V, E)$ 是一個圖形，設 v 是 G 中任意一點，若 G 中找得到以 v 為根點、 t 條分支的子圖 $T_1(v; t)$ ，則 v 是 t -局部可診斷。

由上述內容，我們得知局部診斷演算法如何決定一點的局部可診斷度，從圖 2 可看出為點 v 建立 T_1 的重點只是替點 v 的所有鄰點找到匹配(matching)，我們將在下一章介紹兩種匹配方法。

在 2010 年 Kung 等人[6]提出隨機壞點診斷演算法(Algorithm Diagnose- Random-Fault)，它說明如何在 PMC 模型之下根據診斷得到的症狀以及 T_1 來判斷點的好壞，此演算法的流程相當簡單，我們以 $\sigma(a, b)$ 代表點 a 測試點 b 得到的症狀，假設圖形 G 有一點 v ， v 的分支度是 k 並已在 G 上找到一個 $T_1(v; k)$ 的結構，且 $T_1(v; k)$ 上的壞點數不超過 k ，設 $1 \leq i \leq k$ ，我們要依序檢驗 v 的 k 條分支；若 $(\sigma(y_i, x_i), \sigma(x_i, v)) = (0, 0)$ 代表該分支保證 v 是好點， $(\sigma(y_i, x_i), \sigma(x_i, v)) = (0, 1)$ 代表該分支保證 v 是壞點，檢驗完所有分支並作統計之後，若症狀為 $(0, 0)$ 的分支數量大於等於 $(0, 1)$ 的分支數量，代表 v 是好點，症狀是 $(0, 0)$ 的分支數量小於 $(0, 1)$ 的分支數量，則 v 是壞點。

定理 2[6]. $G(V, E)$ 是一個圖形，設 v 是 G 中任意一點， $t = \deg_G(v)$ ，假設 G 上存在 $T_1(v; t)$ ，則在 $T_1(v; t)$ 之中壞點數量不超過 t 的情況下，隨機壞點診斷演算法能正確判斷 v 是好或壞。

三. 匹配方法

從上述內容，我們運用局部診斷演算法得知一點好壞，須先替要診斷的點建構 T_1 ，建構 T_1 即是找匹配的問題，匹配的方式與圖形結構相關，因此需針對圖形設計適合的方法，才能以規律且較有效率的方式找到匹配，當 T_1 建構完成之後，即以定理 2 的隨機壞點診斷演算法判斷點的好壞，若要診斷整個圖形，則對每個點執行局部診斷演算法，即可完成全圖診斷。

我們提供的第一種匹配方法，是超立方體、交錯立方體以及雙扭立方體皆可使用的方法。假設 n 是維度且 $n \geq 3$ ， i 是圖形中一點的編號且 $0 \leq i \leq 2^n - 1$ ，本匹配方法可為任意一點 i 建構 $T_1(i; n)$ ，時間複雜度為 $O(n)$ ，下述是方法流程，而圖 3 是使用本方法建構 T_1 的例子。

匹配演算法 1.

輸入：圖中任意一點 i

輸出： i 的所有鄰點的匹配

1. **for** $j = 0$ to $n - 2$ **do**
 $AdjacentNode = i$ 的第 j 維鄰點;
 $match[j] = AdjacentNode$ 的第 $j + 1$ 維鄰點(是 $AdjacentNode$ 的匹配點);
2. **end for**
3. $AdjacentNode = i$ 的第 $n - 1$ 維鄰點;
4. $match[n-1] = AdjacentNode$ 的第 0 維鄰點(是 $AdjacentNode$ 的匹配點);
5. **return** $match$;

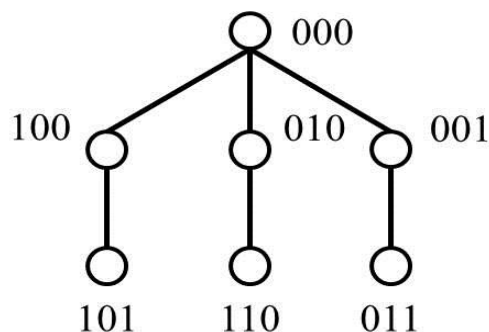


圖 3. 以匹配演算法 1, 替 Q_3 上的點 000 建構的 T_1

第二種匹配方法則是任意一種簡單圖都可使用，隨機簡單圖中大部分為毫無規律性的圖形，不具備像超立方體那般良好的性質，各點的局部可診斷度也不盡相同，假設 $k \geq 0$ ，我們無法得知隨機簡單圖上任意一點 i 的 $T_1(i, k)$ 的 k 其最大值為何，考量到這一點，必須以能找到最大匹配為前提來設計匹配方法；首先我們將尋找最大匹配的問題，簡化為尋找最大二分匹配(Maximum Bipartite Matching)的問題，最大二分匹配只會存在 X 和 Y 兩個互斥點集合， X 集合內的點彼此都不相連且 Y 集合也是如此， X 和 Y 之間則有邊相連，我們把隨機簡單圖中任意一點 i 的所有鄰點當作 X 集合(忽略 X 集合中相連的邊)，而 X 集合中各點的所有鄰點(扣除 X 集合中的點及點 i)則形成 Y 集合(忽略 Y 集合中相連的邊)，如此即可化簡為最大二分匹配問題；接下來我們以知名的匈牙利演算法(Hungarian Algorithm)來解決最大二分匹配問題，匈牙利演算法是 Kuhn[5]基於幾位匈牙利數學家的理論所發展，於 1955 年提出的可解決最大二分匹配問題的演算法，設 X 集合有 n 個點、 Y 集合有 m 個點， $m \geq n$ ，此方法所費時間為 $O(n^2 m)$ [10]。

四. 局部診斷演算法的 MapReduce 設計

在執行局部診斷演算法時，必須知道圖形的結構與測試後得到的症狀，這裡將圖形結構和症狀各自存放到兩個檔案(分別稱為檔案 1 和檔案 2);除了這兩個檔案之外還有第三個檔案

(檔案 3)存放各 map 節點處理範圍須知，檔案 3 的每一行各自記錄一個 map 須處理的點範圍及圖形的最大分支度和圖形總點數。在執行 MapReduce 程式之前，需先將這 3 個檔案存放至 HDFS，然後在執行 MapReduce 程式時，我們先讓 Hadoop 以內建的 TextInputFormat 函式來讀取檔案 3 的內容，每讀一行即形成一組鍵值組對(key-value pair)並做為一個 map 的輸入，map 的工作如下。

演算法 map (key, value)

輸入：鍵值組對<key, value>，key是讀取位置，value是檔案3的一行資料，該行資料包含此map必須處理的點範圍、圖形總點數、圖形最大分支度等等資訊

輸出：鍵值組對<key, value>，key是0，value是壞點編號

1. 分析value，取得起點編號start、終點編號end、總點數number、最大分支度MaxDegree;
2. 從HDFS讀取檔案1，將全部圖形結構內容存入link[number][MaxDegree];
3. 從HDFS讀取檔案2，將全部症狀內容存入syndrome[number][MaxDegree];
4. **for** $i = \text{start to end}$ **do**
 匹配演算法(i);
 counter1 = 症狀為(0,0)的分支數量;
 counter2 = 症狀為(0,1)的分支數量;
5. **if** counter1 < counter2, **then**
 FaultyNodes += i + ","; //i為壞點
6. **End if**
7. **End for**

8. 設key為0，value為FaultyNodes;
9. 輸出鍵值組對<key, value>;
10. 結束

步驟 1~3 為前置作業，取得圖形的相關資訊，步驟 4~7 是為每一點建構 T_1 並判斷點的好壞，最後設定所有 map 輸出的 key 一律為 0，因此所有 map 輸出的鍵值組對會因 key 相同而被 Hadoop 的 combiner 合併成一個一鍵多值的鍵值組對<key, values>，由於只有一組鍵值組對，因此只會建立一個 reduce 並挑一個節點執行它，reduce 的工作如下。

演算法 reduce (key, values)

輸入：鍵值組對<key, values>，key是0，values是每個map輸出的壞點編號所形成的群組

輸出：鍵值組對<key, value>，key是空值，value是全部的壞點編號

1. **for** Text val : values **do**
 AllFaultyNodes += val.toString();
2. **end for**
3. 設 key 為空值，value 為 AllFaultyNodes;
4. 輸出鍵值組對<key, value>;
5. 結束

Reduce 的工作相當單純，它將所有壞點編號合併成一條字串，做為輸出的鍵值組對的 value，key 則設為空值，最後 Hadoop 會將此鍵值組對存入 HDFS 之中，圖 4 為本設計的流程架構圖。

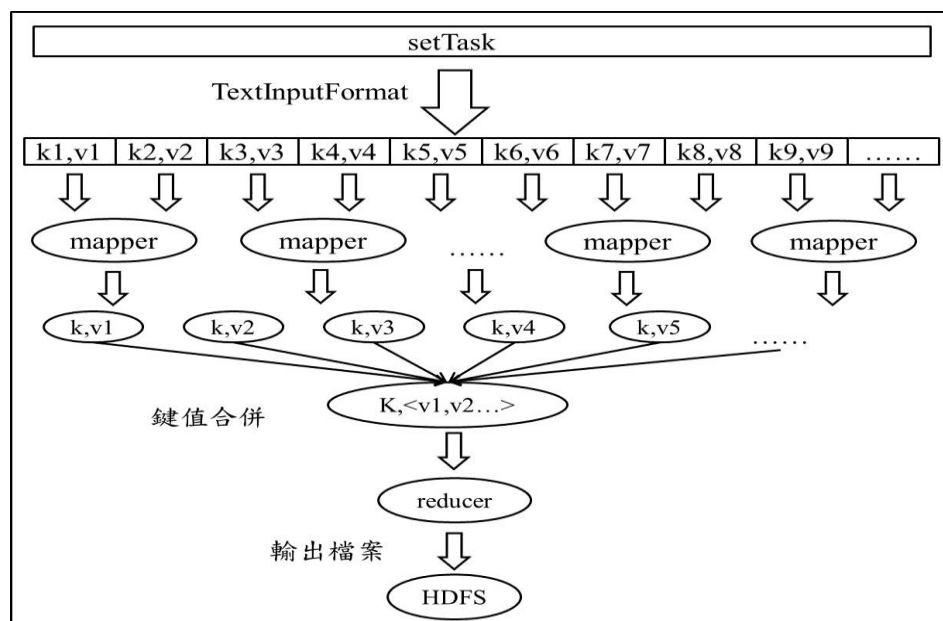


圖 4.局部診斷演算法的 MapReduce 運作流程

五. 實驗結果

用來運算本章節所有實驗的資料節點硬體皆為記憶體 2GB、X3440 2.53GHz 處理器 2 顆、磁碟 269.5GB、作業系統為 Ubuntu 11.10，而所有實驗皆使用 4 個資料節點來進行運算。在此環境上，我們首先以局部診斷演算法來診斷各種維度的超立方體、交錯立方體以及雙扭立方體，並分析它們的運算時間，目前因為受限於系統記憶體大小的緣故，因此本實驗的診斷上

限是到 21 維；第二個實驗是診斷點數與各維度的超立方體相同但分支度略有差異的隨機簡單圖，採隨機取樣的方式取多個樣本進行測試，並計算平均診斷時間，這樣的測試將進行 3 回合，每回合皆設定不同的壞點數量，最後對結果進行分析。

5.1 超立方體、交錯立方體、雙扭立方體診斷時間

超立方體		
維度	點數	時間(秒)
10	1024	22
11	2048	22
12	4096	23
13	8192	22
14	16384	23
15	32768	22
16	65536	22
17	131072	25
18	262144	25
19	524288	28
20	1048576	43
21	2097152	94

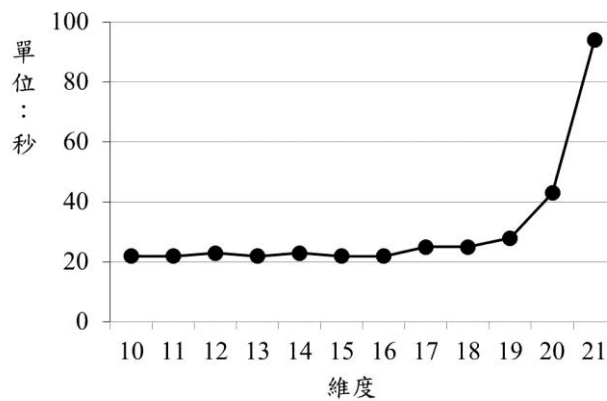


表 2. 以局部診斷演算法，診斷各種維度的超立方體所耗費的時間

交錯立方體		
維度	點數	時間(秒)
10	1024	22
11	2048	24
12	4096	22
13	8192	24
14	16384	22
15	32768	23
16	65536	22
17	131072	25
18	262144	25
19	524288	28
20	1048576	44
21	2097152	98

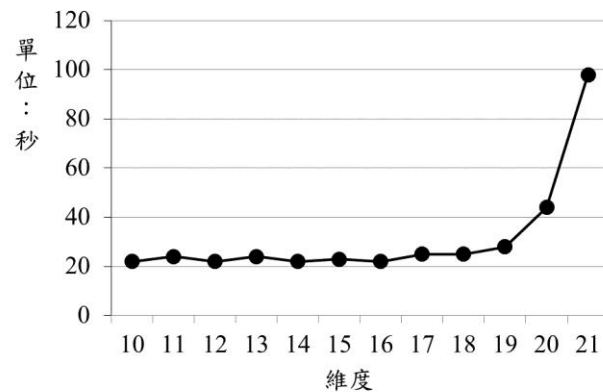


表 3. 以局部診斷演算法，診斷各種維度的交錯立方體所耗費的時間

雙扭立方體		
維度	點數	時間(秒)
11	2048	22
13	8192	22
15	32768	22
17	131072	25
19	524288	28
21	2097152	96

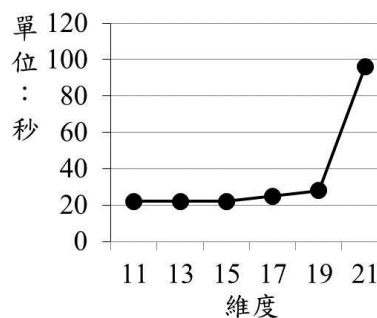


表 4. 以局部診斷演算法，診斷各種維度的雙扭立方體所耗費的時間

先觀察表 2，可發現當維度 ≤ 18 時，都在相當快速且接近的時間完成，表示該情況下 map 運算時間非常微小，大部分時間為 Hadoop 其他固定流程所耗費，而維度是 18 時點數為 262144，換算下來每個資料節點須運算 65536 個點，可得知當資料節點負責運算的點數小於 65536 時，都可以相當快速且接近的速度完成，至於診斷時間全落於 22 秒~25 秒之間，彼此之間出現 1~3 秒的些微落差，這是因為在分配 map 和 reduce 的工作以及結果輸出時，受到網路速度所影響。接下來直到維度是 19 時 mapper 的運算時間開始緩緩上升，維度是 20 時，仍以不到一分鐘的速度完成診斷一百萬個點，最後當維度達到 21 時，才使診斷速度有了較明顯的變慢，可發現此時維度 21 的點數是維度 20 的一倍，運算時間也與之成正比，大約上升了一倍；再來觀察表 3 和表 4，可發現交錯立方體與雙扭立方體的診斷時間都與超立方體相似，雖然彼此的結構不相同，但因使用同一種匹配方法，使得它們的診斷時間一定會非常接近。整體而言，由於這三種圖形都具有規則的連線的特性，因此為各點建構 T_1 的匹配方法能夠設計得非常簡單且快速。

5.2 隨機簡單圖診斷時間與可診斷性

這個實驗我們以局部診斷演算法來診斷隨機簡單圖，所有樣本皆根據維度在 10 維~21 維之間的超立方體的點數和分支度來產生，但分支度偏移值為 1，我們進行三回合的測試，第一回合將壞點數設定為圖形總點數的 10%，第二回合為 20%，第三回合則是 30%，藉此實驗來觀察超立方體與無規則的圖形以局部診斷演算

法來診斷的時間差異性，並藉以體會隨機圖形的可診斷性，診斷時間如下表。

數據顯示，在點數 ≤ 131072 的時候，只需二十幾秒即可完成診斷，map 運算時間很小，直到點數為 262144 時，時間才有較明顯的升高，與此點數相對的是 18 維的超立方體，但在上個實驗 18 維的超立方體依然能快速診斷，彼此時間上升的轉捩點相差 1 個維度，並且發現在 19 維之後的診斷時間約為超立方體的 1 倍。會造成如此差距是因為隨機簡單圖的匹配方法較為複雜，因為匹配方法不同而造成的時間差距完全體現於數據之中，由此實驗可發現局部診斷演算法對於結構良好的圖形能有更快的診斷速度；而在這三個回合的測試中，當點數 ≤ 262144 時，它們的平均時間完全一樣，這是因為診斷時間都非常快速，時間的飄移範圍很小所致，直到點數達到一百萬點以上時，才讓彼此出現數秒的差距，具有相當程度的穩定性。另外，當壞點數量是圖形總點數的 10% 時，所有樣本均被成功診斷出所有壞點，假設 v 的分支度是 d ，經過觀察各樣本的連線後發現，任意一點 v 的各個鄰點的所有鄰點相同率不高，因此 v 的各個鄰點能輕易的找到配對，可為 v 建構出 k 值接近 d 的 $T_1(v; k)$ ，且因壞點分佈平均，使 T_1 上的壞點數量不容易超過 k ；接下來壞點數達到 20% 時，出現極少數無法被完全診斷的樣本，觀察這些樣本後，發現它們其實只有少數幾點無法診斷，最後在第三回合將壞點數量提升到 30% 時，開始出現大量無法被完全診斷的樣本，每個樣本都已出現許多點無法被正確診斷的情形。

隨機簡單圖								
點數	分支度	隨機取樣數	第一回合平均時間(秒)	第二回合平均時間(秒)	第三回合平均時間(秒)	第一回合無法診斷的樣本數	第二回合無法診斷的樣本數	第三回合無法診斷的樣本數
1024	9~11	10	22	22	22	0	0	4
2048	10~12	10	22	22	22	0	0	3
4096	11~13	10	22	22	22	0	1	4
8192	12~14	10	22	22	22	0	0	5
16384	13~15	15	22	22	22	0	0	7
32768	14~16	15	22	22	22	0	0	6
65536	15~17	15	25	25	25	0	0	7
131072	16~18	20	26	26	26	0	0	10
262144	17~19	20	32	32	32	0	1	9
524288	18~20	20	43	43	44	0	0	8
1048576	19~21	30	71	70	72	0	0	16
2097152	20~22	30	146	144	147	0	1	13

表 5. 以局部診斷演算法，診斷複數取樣的隨機簡單圖的平均時間，及無法完全診斷的樣本數量

六. 結論

局部診斷演算法具有泛用性，它能診斷任何一種簡單圖，並存在最大匹配方法以對圖上的每一點建構 T_1 ，因此使我們能決定任意一種隨機簡單圖的局部可診斷度，在診斷速度方面，光使用4個資料節點就足以在極短的時間內診斷規模龐大的超立方體、交錯立方體以及雙扭立方體等結構，且當隨機簡單圖的點數達到200萬點時，也只需花費數分鐘的時間即可完成診斷，具有即時性，若我們能以大型的Hadoop平台使用大量資料節點進行運算，將會得到更快速的效益，此診斷演算法也會隨著圖形結構的好壞而產生效率上的差異；在診斷能力方面，當壞點數量非常高時，也存在著能被正確診斷出所有壞點的樣本。

參考文獻

- [1] T. Araki, "Optimal adaptive fault diagnosis of cubic Hamiltonian graphs," *Proceedings of the 7th International Symposium on Parallel Architectures 2004, Proc. of I-SPAN 2004*, Hong-Kong, May. 2004, pp. 162-167.
- [2] K. Efe, "The crossed cube architecture for parallel computing," *IEEE Trans. Parallel and Distributed Systems*, vol. 3, no. 5, pp. 513-524, Sept. 1992.
- [3] P. A. J. Hilbers and M. R. J. Koopman and J. L. A. van de Snepscheut, "The twisted cube," *Lecture Notes in Computer Science*, vol. 258, pp. 152-159, Jun. 1987.
- [4] G.-H. Hus and J. J. M. Tan, "A local diagnosability measure for multiprocessor systems," *IEEE Trans. Parallel and Distributed Systems*, vol. 18, no. 5, pp. 598-607, 2007.
- [5] H. W. Kuhn, "The hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, pp. 83-97, Mar. 1955.
- [6] T.-L. Kung, H.-C. Chen, and J. J. M. Tan, "On the faulty sensor identification algorithm of wireless sensor networks under the PMC diagnosis model," *IEEE Trans. Networked Computing and Advanced Information Management, NCM*, vol. I, Seoul, South Korea, Aug. 2010, pp. 657-661.
- [7] S. Lattanzi, B. Moseley, S. Suri, and S. Vassilvitskii, "Filtering: a method for solving graph problems in mapreduce," *Federated Computing Research Conference, FCRC*, San Jose, California, 4-11 Jun. 2011, pp. 85-94.
- [8] P.-C. Li, Z.-D. Liu, and P.-L. Lai, "Three-round adaptive diagnosis in crossed cubes," *Proceeding of the 28th Workshop on Combinatorial Mathematics and Computation Theory*, Penghu, Taiwan, May 2011, pp. 254-259.
- [9] F.P. Preparata, G. Metze, and R.T. Chien, "On the connection assignment problem of diagnosis systems," *IEEE Trans. Electronic Computers*, vol. 16, no. 12, pp. 848-854, Dec. 1967.
- [10] R. Z. Ríos-Mercado, Y. A. Rios-solis, "Just-in-Time Systems," Springer Verlag, 2011.
- [11] J. Tan, X. Pan, S. Kavulya, E. Marinelli, R. Gandhi, P. Narasimhan, "Kahuna: problem diagnosis for MapReduce-based cloud computing environments," *Network Operations and Management Symposium, NOMS*, Osaka, Japan, 19-23 Apr. 2010, pp. 112 - 119.
- [12] X. Yang and Y.Y. Tang, "A $(4n - 9)/3$ diagnosis algorithm on n -dimensional cube network," *Information Sciences*, vol. 177, no. 8, pp. 1771-1781, 2007.
- [13] W. Zhao, H. Ma, and Q. He, "Parallel k -means clustering based on MapReduce," *Lecture Notes in Computer Science*, vol. 5931, Beijing, China, 1-4 Dec. 2009, pp. 674-679.